Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning

Trabajo de grado · Modalidad Investigación

Yeison Adrián Cáceres Torres Ricardo Andrés Pérez Porras

Director: Gilberto Javier Díaz Toro, Ph.D.

Universidad Industrial de Santander
Escuela de Ingeniería de Sistemas e Informática

CAGE

Contenido

1. Contexto
2. Problema
3. Estado del arte
4. Pregunta y objetivos
5. Fundamentos
6. Metodología
7. Resultados
8. Conclusiones

1 Contexto
2 Problema
3 Estado del arte
4 Pregunta y objetivos
5 Fundamentos
6 Metodología
7 Resultados
8 Conclusiones

La energía limita cuánto puede escalar un nodo CPU–GPU



Nodo heterogéneo

CPU y GPU comparten la carga científica



Potencia y energía

Cada dispositivo consume según su frecuencia y su actividad



Costo y disponibilidad

La energía incide en los costos operativos y en la sostenibilidad

DVFS ajusta voltaje y frecuencia en el hardware, pero
el beneficio depende de lo que hace la aplicación en cada fase

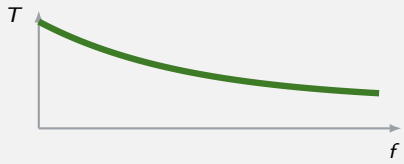
1 Contexto
2 Problema
3 Estado del arte
4 Pregunta y objetivos
5 Fundamentos
6 Metodología
7 Resultados
8 Conclusiones

Una misma aplicación alterna fases que responden distinto a la frecuencia

Una aplicación

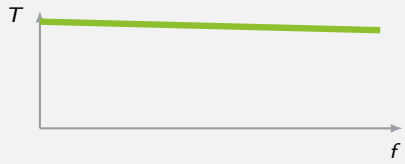


Compute-bound



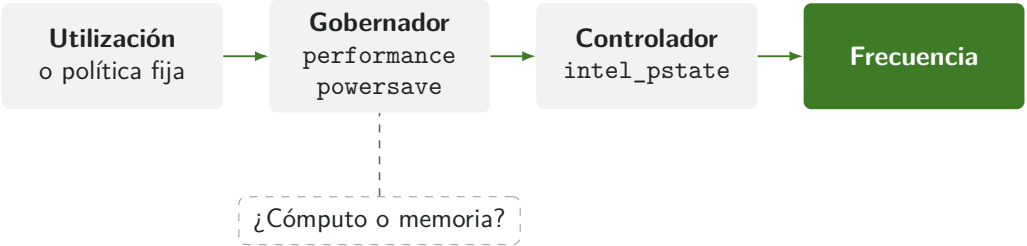
Más frecuencia, menos tiempo

Memory-bound



Más frecuencia, mismo tiempo

Los gobernadores nativos no consideran qué limita la fase



No es una entrada de la decisión

Fuente: Linux Kernel Documentation, *intel_pstate CPU Performance Scaling Driver* y *CPU Performance Scaling*; Hebbar y Milenković (2022).

1 Contexto
2 Problema
3 Estado del arte
4 Pregunta y objetivos
5 Fundamentos
6 Metodología
7 Resultados
8 Conclusiones

La literatura resuelve el problema por partes



Calore et al.

APORTA
DVFS por región, en CPU y GPU

DEJA ABIERTO
Decisión ligada a funciones instrumentadas

Guerreiro Ali

APORTA
Predicen la respuesta a DVFS en GPU

DEJA ABIERTO
Caracterización previa; solo GPU

MCBound

APORTA
Clasifica el régimen de la carga

DEJA ABIERTO
No actúa sobre la frecuencia

Simsek DRLCap

APORTA
Ajustan la frecuencia de GPU en ejecución

DEJA ABIERTO
Instrumentación; solo GPU

Este trabajo

Clasifica la fase y actúa en CPU y GPU, en ejecución, sin instrumentar la aplicación

1 Contexto
2 Problema
3 Estado del arte
4 Pregunta y objetivos
5 Fundamentos
6 Metodología
7 Resultados
8 Conclusiones

Pregunta de investigación

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

Objetivo general

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS) en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

Cuatro objetivos específicos, cuatro fases

1

Caracterizar cargas y consumo bajo distintas frecuencias con contadores e interfaces de potencia estándar

2

Entrenar y validar clasificadores ligeros de fase con baja latencia de inferencia

3

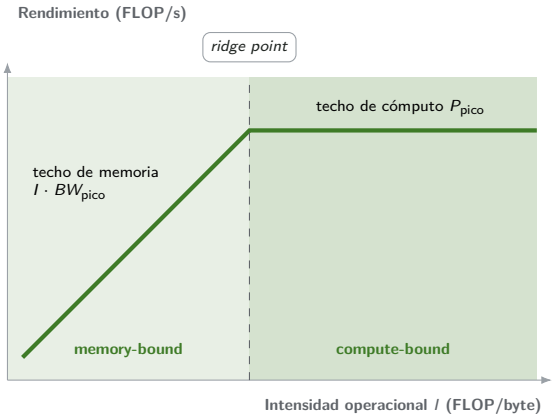
Desarrollar un *daemon* en espacio de usuario que infiera la fase y aplique DVFS

4

Evaluar con EDP si el ahorro compensa la sobrecarga frente a los gobernadores nativos

1 Contexto
2 Problema
3 Estado del arte
4 Pregunta y objetivos
5 Fundamentos
6 Metodología
7 Resultados
8 Conclusiones

Roofline: la intensidad operacional define la etiqueta



Adaptado de Williams et al. (2009).

$$P = \min\left(P_{\text{pico}},\ I \cdot BW_{\text{pico}}\right)$$

$$I_i = \frac{\text{FLOPs}_i}{B_i^{\text{DRAM}}} \qquad I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}}$$

$$I < I_{\text{ridge}} \Rightarrow \textbf{memory-bound}$$
$$I \geq I_{\text{ridge}} \Rightarrow \textbf{compute-bound}$$

Los techos se calibran en el nodo, por nivel de frecuencia y precisión.

Bajar la frecuencia reduce la potencia, no siempre la energía

Potencia

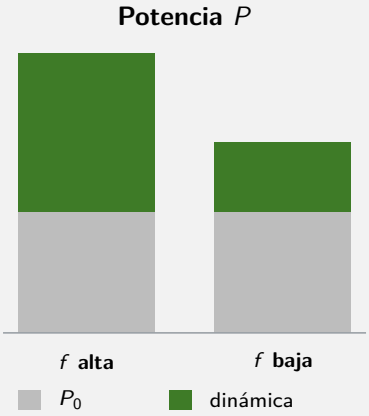
$$P = \underbrace{P_0}_{\text{piso}} + \underbrace{\alpha\, C\, V^2 f}_{\text{dinámica}}$$

Energía

$$E = P \cdot T$$

EDP

$$\text{EDP} = E \cdot T$$



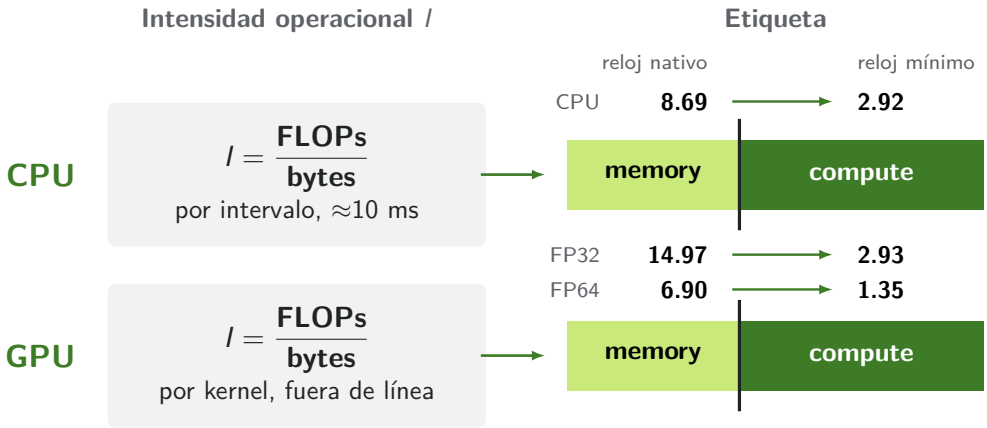
1 Contexto
2 Problema
3 Estado del arte
4 Pregunta y objetivos
5 Fundamentos
6 Metodología
7 Resultados
8 Conclusiones

Fase 1: cada carga se mide en diez niveles de frecuencia



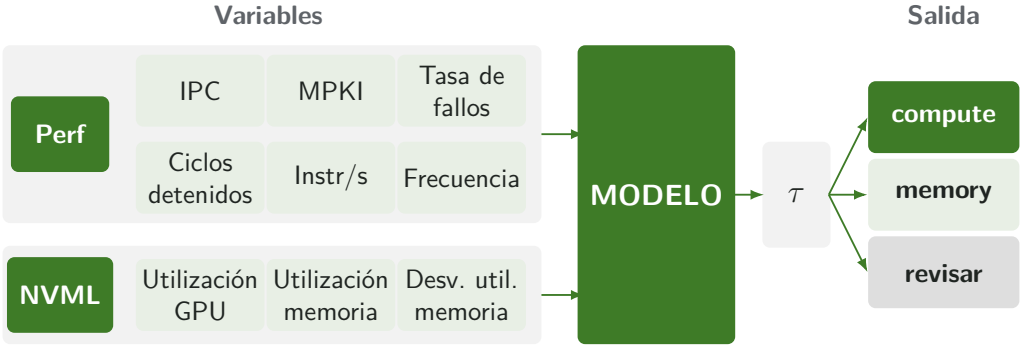
REF: estado nativo del dispositivo. CPU con turbo desactivado. Un solo nodo, exclusivo vía Slurm.

La etiqueta compara la intensidad medida con el *ridge* calibrado



Ridge en FLOP/B, calibrado en el nodo por nivel de frecuencia y precisión. Cada flecha va del reloj nativo al reloj mínimo.

Fase 2: un clasificador ligero aproxima la etiqueta con señales



Prohibidas como entrada

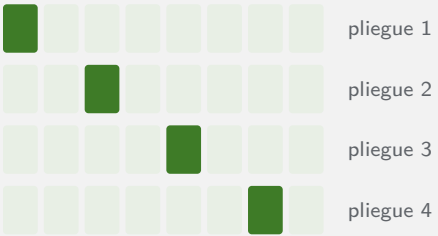
I, FLOPs, bytes, *ridge*, etiqueta

Modelos evaluados

Mayoritario, árbol, regresión logística, *random forest*, *extra trees*, XGBoost

La validación retira familias completas y la política se deriva aparte

Dejando una familia fuera (LOFO)



$$\hat{y}_i = \begin{cases} \text{memory-bound}, & p_i \geq 0.5 \ \wedge \ q_i \geq \tau \\ \text{compute-bound}, & p_i < 0.5 \ \wedge \ q_i \geq \tau \\ \textit{revisar}, & q_i < \tau \end{cases}$$

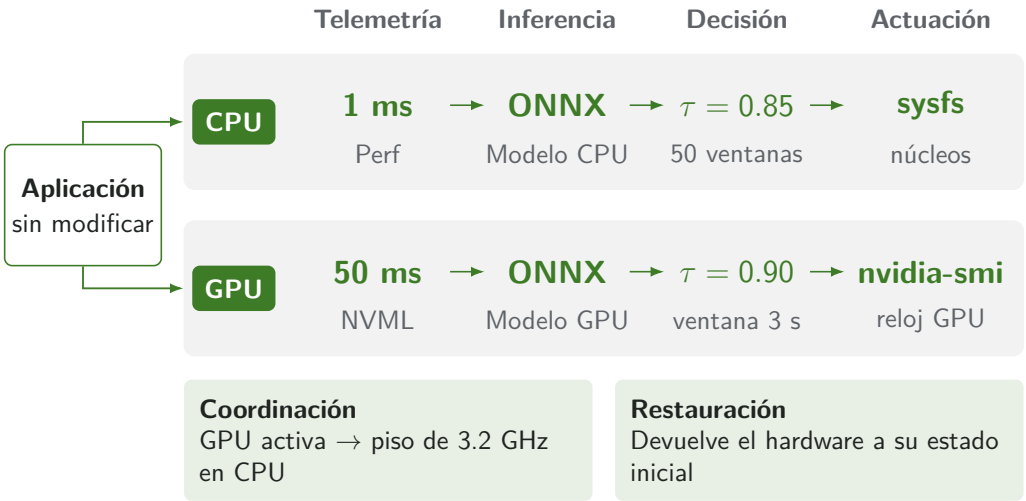
Métrica de decisión: exactitud balanceada por celda

aciertos promediados por celda (familia $\times$ clase)

τ elegido sin la familia retirada

Política: por clase, el nivel con mejor EDP frente a la base

Fase 3: dos procesos C++ cierran el lazo en espacio de usuario



Fase 4: brazos comparables y EDP CPU+GPU frente a la base nativa

	Muestrea	Decide	Escribe
Base sin agente	○	○	○
Sombra con agente	●	●	○
Activo con agente	●	●	●
F1 fijo sin agente, reloj GPU fijo	○	○	●

$$\mathrm{EDP}_{\mathrm{CPU+GPU}} = (E_{\mathrm{pkg0}} + E_{\mathrm{pkg1}} + E_{\mathrm{GPU}}) \cdot T$$

Bloques aleatorizados

Turbo de CPU desactivado; en la base, la GPU queda en gestión automática del controlador.

Seis escenarios, de aplicaciones armadas a aplicaciones externas

Aplicaciones armadas

Matriz inicial

Compuestas A y B

E-A

Memoria larga · vistas

E-B

Memoria larga · inéditas

E-C

E-A + cómputo sensible al reloj

Aplicaciones externas

LAMMPS

Aplicación externa · 4 entradas

CloverLeaf

Aplicación externa continua

1 Contexto
2 Problema
3 Estado del arte
4 Pregunta y objetivos
5 Fundamentos
6 Metodología
7 Resultados
8 Conclusiones

Fase 1: el conjunto de datos cubre ambos regímenes

42 458

intervalos CPU · 30 familias

tope de 1000 por familia y clase

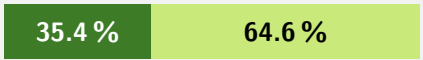


21 350 compute · 21 108 memory

483

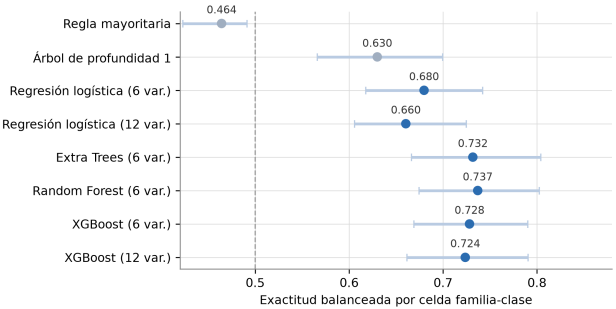
corridas GPU · 16 familias

19 kernels × 10 niveles de reloj



171 compute · 312 memory

Fase 2: en CPU los modelos empatan; se eligió el más rápido



Exactitud balanceada por celda **empatada**

Latencia por intervalo

243 µs XGBoost

5.3 ms Random Forest, Extra Trees

Hiperparámetros de XGBoost

+0.019, sin diferencia clara

Familia retirada. Latencia en Python, un hilo.

XGBoost: el modelo final de CPU

Configuración

Modelo	**XGBoost**
Árboles	**100 · prof. 6**
Entradas	**6 de Perf**
Pesos	**por celda**
Salida	**P(*memory*)**
Validación	**familia retirada**

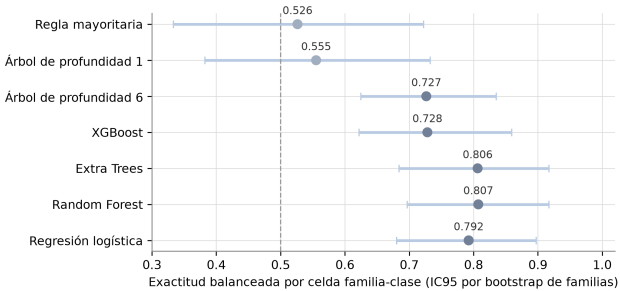
Métricas sin abstención

0.769 Exactitud común	**0.714** Sens. *compute*	**0.806** Sens. *memory*
0.760 F1 macro	**0.842** AUC-ROC	**0.520** Matthews

0.728 por celda (al azar: 0.874)

0.738 decide 72.2 % con $\tau = 0.85$

En GPU los modelos también empatan



Exactitud balanceada por celda **empatada**

Latencia por predicción

4.7 ms **4.8 ms**

Random Forest Extra Trees

Hiperparámetros

empeoran 4 de 5 modelos

Familia retirada. Latencia mediana por fila en Python, nodo de destino, mismas variables.

En GPU se eligió el modelo que no depende del reloj que el agente modifica

Configuración

Modelo	***Random forest***
Entradas	**3 de NVML**
Sin	**reloj ni potencia**
Pesos	**por celda**
Salida	**P(*memory*)**
Validación	**familia retirada**

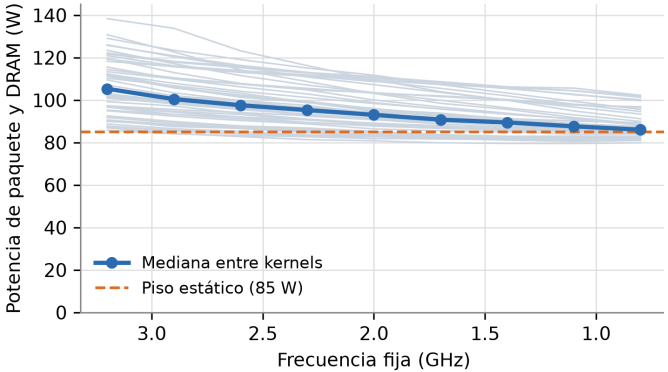
Métricas sin abstención

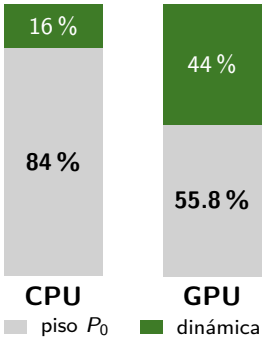
0.892 Exactitud común	**0.825** Sens. *compute*	**0.929** Sens. *memory*
0.881 F1 macro	**0.913** AUC-ROC	**0.763** Matthews

0.819 por celda (sin abstención)

0.900 con $\tau = 0.90$ cobertura 65.7 %

Fase 3: bajar el reloj de CPU casi no baja la potencia





Potencia al reloj más alto.

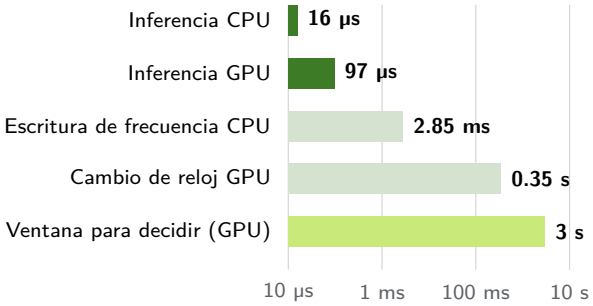
Izquierda: potencia de paquete y DRAM de CPU con la frecuencia fija de los núcleos delegados; cada línea gris es un kernel.

El piso de potencia decide: no actuar en CPU y F1 en GPU *memory-bound*

	compute-bound	**memory-bound**
CPU	No actuar la mejor opción gana solo 0.6 %	No actuar bajar a F1 empeora 5.4 %
GPU	No actuar bajar a F1 gana 2.1 %, pero no siempre	**F1 = 1260 MHz** +8.9 % EDP GPU 6.5 % en programas nuevos

Mejora de EDP frente a la base. Se actúa solo si supera el 1 % con evidencia.

Inferir cuesta microsegundos; el proceso de CPU completo sí cuesta



Escala logarítmica de tiempo.

Sin costo — El agente de GPU observando deja el EDP CPU+GPU igual a la base

6–14 W — El agente de CPU gasta esto de más, sin actuar

+4–6 % — Por eso el agente de CPU empeora el EDP CPU+GPU

Fase 4: el agente de GPU gana con fases de memoria largas

Aplicaciones armadas — Cambio del EDP CPU+GPU frente a la base

Matriz inicial +0.2–1.0 %
compuestas A y B

E-A −3.8 %
F1 fijo logra lo mismo

E-B +1.6 %
familias inéditas: se abstuvo

E-C −2.4 %
supera a F1 fijo (6 de 6 bloques)

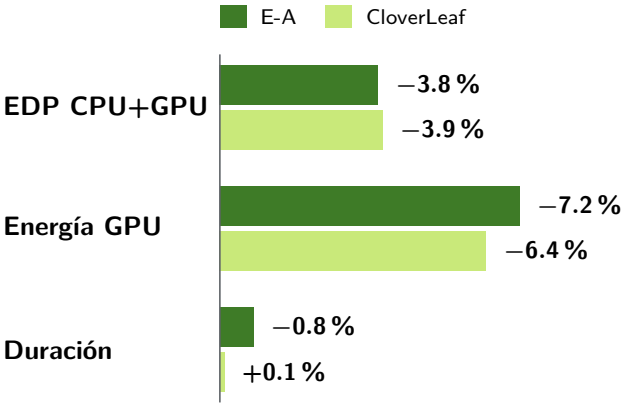
Aplicaciones externas

LAMMPS +0.3–0.9 %
nunca aplicó F1

CloverLeaf −3.9 %
5 de 5 bloques

Con fases de memoria largas, el EDP CPU+GPU baja 3.8–3.9 %

Cambio frente a la base



5 de 5 bloques mejoran en los dos escenarios

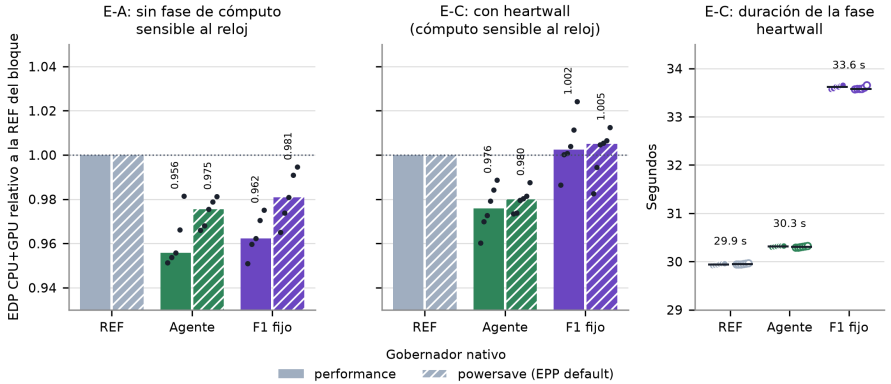
Observar sin actuar no ahorra: el ahorro viene de actuar

Con *powersave*: −2.6 % y −3.2 %

Pero aquí un reloj fijo en F1 logra lo mismo

E-A: compuesta con familias vistas. CloverLeaf: aplicación externa. La CPU es ~64 % de la energía medida y no se modifica.

¿Clasificar o fijar F1? Clasificar gana solo si hay cómputo sensible al reloj



E-A · empatan
−0.2 % frente a F1 fijo

E-C · gana el agente
−2.8 %, 6 de 6 bloques

E-B · gana F1 fijo
el agente se abstuvo

E-C es una prueba de mecanismo: escenario construido con el kernel más favorable y una familia vista.

1 Contexto
2 Problema
3 Estado del arte
4 Pregunta y objetivos
5 Fundamentos
6 Metodología
7 Resultados
8 Conclusiones

Tres hallazgos

1. **84 % vs 55.8 %**
   El piso de potencia decide dónde vale la pena el DVFS
   En CPU casi toda la potencia es piso y no hay margen; en GPU sí

2. **0.819**
   Señales ligeras aproximan el régimen Roofline sin perfilar
   En familias no vistas, con inferencia de 97 µs; perfilar con Nsight alarga 60 veces la ejecución

3. **−2.8 %**
   Clasificar por fase gana cuando hay cómputo sensible al reloj
   Frente a un reloj fijo en E-C; donde no hay esa fase (E-A), una regla fija rinde igual

Respuesta a la pregunta de investigación

	GPU	CPU
¿Reduce el consumo?	**Sí:** -3.8 / $-3.9\,\%$ de EDP CPU+GPU solo GPU: -8.0 / $-6.3\,\%$; con memoria larga	**No:** con un piso de 84 % no hay margen
¿La inferencia degrada el rendimiento?	**No:** 97 µs por decisión; la duración no cambia	**No es inferir** (16 µs): observar sube 5–9 % la energía de CPU
¿Frente a los gobernadores nativos?	**Se mantiene** con *performance* y *powersave*	Empeora el EDP con ambos

Recomendación: desplegar solo el proceso de GPU

Limitaciones: qué no afirma este trabajo

Un solo nodo

Xeon Gold 5315Y + A100. No se generaliza a otras CPU, GPU ni controladores

Energía medida

Paquetes de CPU y GPU. No incluye DRAM, placa, fuente ni ventilación: no es la energía del nodo

Pocas repeticiones

Solo 5 por escenario: no bastan para una prueba concluyente. E-C se construyó para probar el mecanismo

CPU acotada

Solo la frecuencia de 6 núcleos delegados, sin turbo ni *uncore*. No es la CPU completa

Gobernadores

performance y *powersave*; *ondemand* y *schedutil* no existen con `intel_pstate`

Modelo de GPU

Entrenado por corrida y aplicado en ventanas de 3 s; con fases cortas o familias nuevas se abstiene

Trabajo futuro

- Más familias externas *memory-bound* en GPU, evaluadas una sola vez con el modelo congelado
- Repetir base, agente y F1 fijo en aplicaciones externas con cómputo sensible al reloj
- Reentrenar el modelo de GPU por ventana, con fronteras de fase de CUPTI
- Reducir primero el costo de observar en CPU, antes de revisar su política
- Probar en otras arquitecturas con un piso de potencia de CPU más bajo, donde el DVFS de CPU sí tenga margen
- Actuar también sobre la frecuencia del *uncore* y de la memoria en CPU, no solo sobre la de los núcleos

Gracias

Yeison Adrián Cáceres Torres · Ricardo Andrés Pérez Porras
Director: Gilberto Javier Díaz Toro, Ph.D.

Repositorio: `github.com/AdrianCCRS/hyperion`





Índice del anexo

Fase 1 · Medición
A1 Composición de clase por familia
A2 Validación de la medición

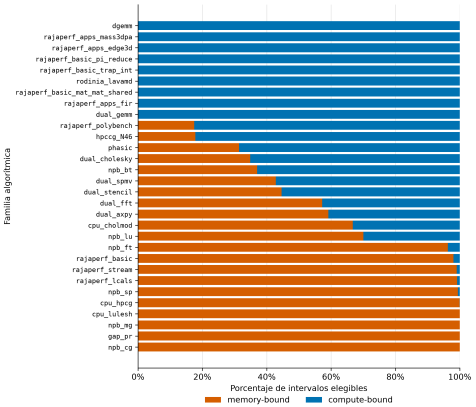
Fase 2 · Clasificador CPU
A3 Matrices de confusión
A4 Resultado por familia
A5 Perfiles gemelos
A6 Abstención

Fase 2 · Clasificador GPU
A7 Matrices de confusión
A8 Invarianza al reloj

Fase 3 · Política y agente
A9 Política CPU
A10 Energía relativa CPU
A11 Política GPU
A12 Energía relativa GPU
A13 Verificación del agente

Fase 4 · Evaluación
A14 Aplicaciones compuestas
A15 Matriz inicial
A16 Potencial de la GPU
A17 Escenario E
A18 CloverLeaf
A19 LAMMPS

A1 · Composición de fase por familia en CPU



1 309 755 intervalos elegibles (91.9 %): 40.2 % *compute*, 59.8 % *memory*

La fracción *compute* sube de 29.0 % (nativo) a 46.4 % (F8) porque el *ridge* baja

Familias mixtas (npb_bt, npb_lu, cholmod, phasic. . .) aportan ejemplos cerca del *ridge*

A2 · Validación de la medición: techos y frecuencia efectiva

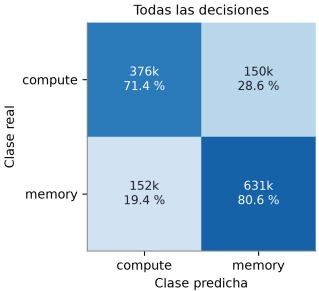
Ancho de banda
Calibrado 58.4–58.8 GB/s frente a 64.0–66.8 GB/s de Intel Advisor (8–13 % por debajo)

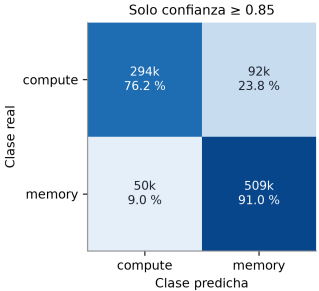
Techo de cómputo
417.8–555 GFLOP/s, consistente con 577.7 GFLOP/s de Advisor en los mismos seis núcleos

***Ridge* nativo**
Advisor lo ubica entre 7.1 y 9.5 FLOP/B; la campaña usó 8.69 FLOP/B

Frecuencia efectiva
Restringir también los hermanos SMT llevó la razón de rendimiento F0/F8 de 0.995 a 3.78 (esperado ≈ 4)

A3 · Matrices de confusión y métricas del clasificador CPU



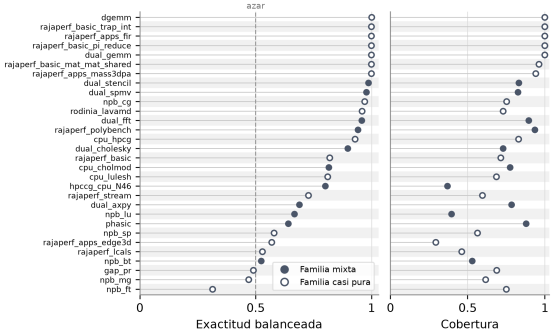


Exactitud 0.769 · F1 macro 0.760 · AUC 0.842 · MCC 0.520

Sensibilidad: *compute* 0.714, *memory* 0.806

Con $\tau = 0.85$ (derecha), F1 macro 0.842, pero refleja sobre todo qué intervalos quedan fuera

A4 · El resultado en CPU varía mucho entre familias

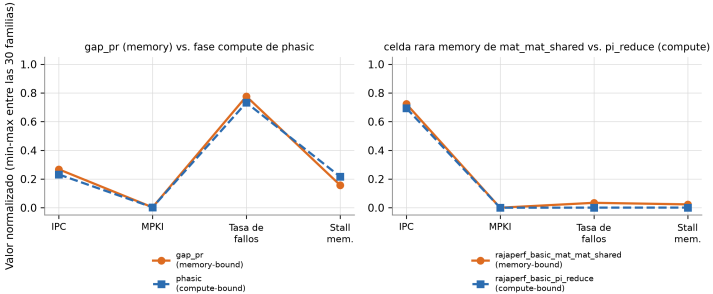


14 familias superan 0.9 de exactitud

5 quedan por debajo de 0.55; npb_ft en **0.314**

Las peores tienen un perfil de contadores igual al de familias de la clase contraria

A5 · El límite está en la información: perfiles gemelos

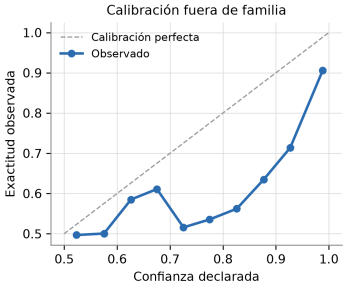


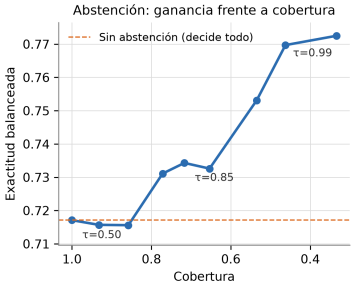
gap_pr (*memory*) y la fase *compute* de phasic: MPKI 0.82 vs 0.94, IPC 1.11 vs 0.97

kNN sin ajuste: 0.684; curva de aprendizaje 0.615 → 0.718 (3 a 23 familias)

Con *I* como entrada se llega a 0.9989: la brecha es información que las seis señales no transportan

A6 · Abstención en CPU: $\tau = 0.85$ decide el 72.2 %



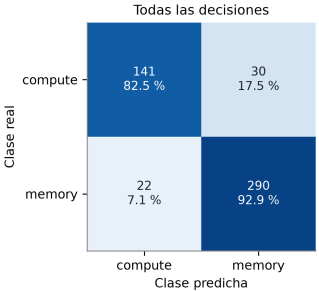


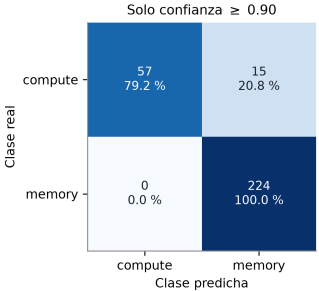
Sobre lo decidido sube solo de 0.728 a 0.738; abstenerse al azar no mejora

Sobreconfianza fuera de familia: confianza media 0.890, exactitud 0.764 (ECE 0.126)

Con confianza ≥ 0.9, npb_ft acierta 32 % y npb_mg 35 %

A7 · Matrices de confusión del *random forest* de GPU



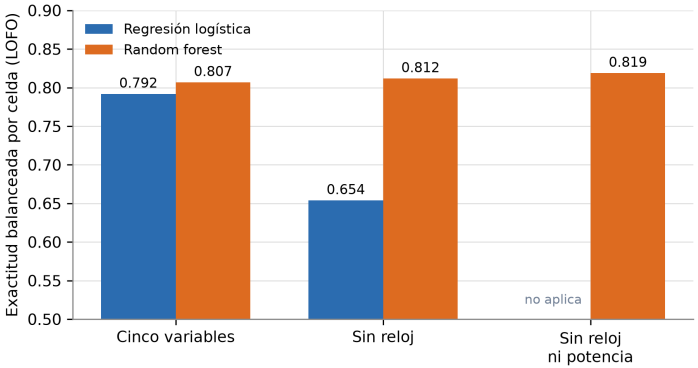


Exactitud 0.892 · F1 macro 0.881 · AUC 0.913 · MCC 0.763

Con $\tau = 0.90$: 0.900, cobertura por celda 65.7 %; 0 errores en *memory*

De 239 corridas que activarían F1, 224 son *memory* (precisión 0.937)

A8 · Por qué el modelo de GPU no usa el reloj ni la potencia

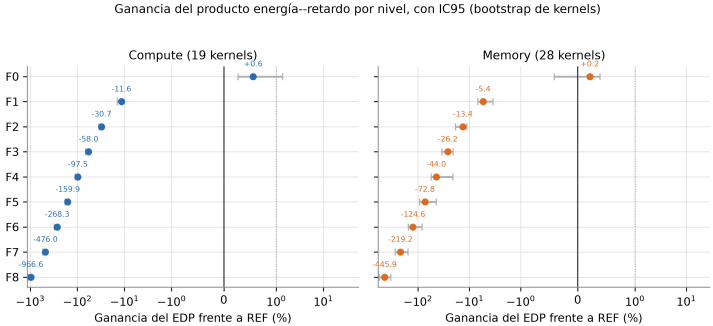


Al fijar el reloj en 1410 MHz, la regresión logística cambia de clase el 26.7 % de sus predicciones

Sin el reloj, la regresión logística baja a 0.654; el *random forest* no pierde nada

Frente a *extra trees*: mayor sensibilidad (0.809 y 0.891 vs 0.749 y 0.855), menor variación entre semillas y *dual_cholesky* 0.51 vs 0.15

A9 · Política CPU: no actuar en ninguna clase

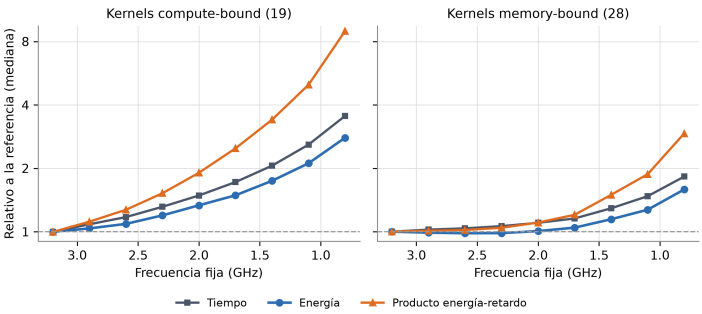


Cómputo: F0 mejora 0.56 %, menos que el efecto mínimo de 1 %

Memoria: F1 empeora 5.4 % (IC95 3.4 a 6.8)

Un oráculo por kernel ganaría 2.2 % en promedio; 22 de 47 kernels bajo 1 %

A10 · En CPU el tiempo crece más de lo que cae la potencia

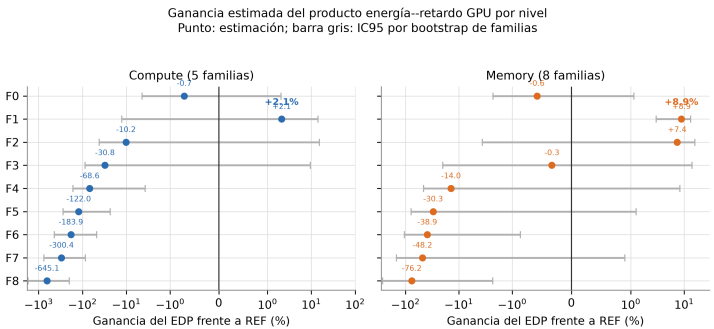


La energía de memoria se mantiene cerca de la referencia hasta ~2 GHz

Con $P(f) = P_0 + c\,f^k$, bajar el reloj mejora el EDP solo si $\beta < \beta_{\text{crít}}$

$\beta_{\text{crít}}$: mediana 0.14, máximo 0.33; el cómputo denso tiene $\beta \approx 1$

A11 · Política GPU: no actuar en cómputo, F1 en memoria

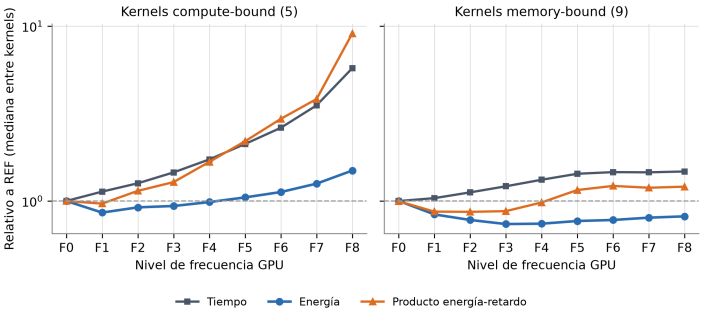


Cómputo: F1 $+2.1\,\%$ (IC95 -12.7 a 14.2; $p = 0.438$) $\rightarrow$ no actuar

Memoria: F1 $+8.9\,\%$ (IC95 3.0 a 13.3; $p = 0.020$), estimación interna; con la familia fuera, $6.5\,\%$ (IC95 -4.0 a 13.1)

rodinia_gaussian pierde $27.3\,\%$: $\beta = 0.941$ pese a ser *memory-bound*

A12 · En GPU la energía de memoria cae hasta F3



Piso estático $P_0 = 38.5$ W: 55.8 % de la potencia al reloj más alto (CPU: 84 %)

β mediano: *compute* 0.837, *memory* 0.189

EDP < 1 en F1–F2 para *memory*; desde F5 domina el tiempo

A13 · Verificación del agente antes de la evaluación

Exportación ONNX

Diferencia máx. de probabilidad 3×10^{-7} (CPU) y 9×10^{-8} (GPU); ninguna etiqueta cambia

Latencias

Inferencia 16.4 µs (CPU) y 97 µs (GPU); escritura CPU 2.85 ms en paralelo; cambio de reloj GPU 0.35–0.37 s

Restauración

Ante SIGTERM o SIGINT devolvió reloj GPU, turbo y rango de CPU al estado inicial, código de salida 0

Coordinación

Con ambos procesos activos, el piso de CPU de 3.2 GHz se respetó en el 99.2 % de las muestras

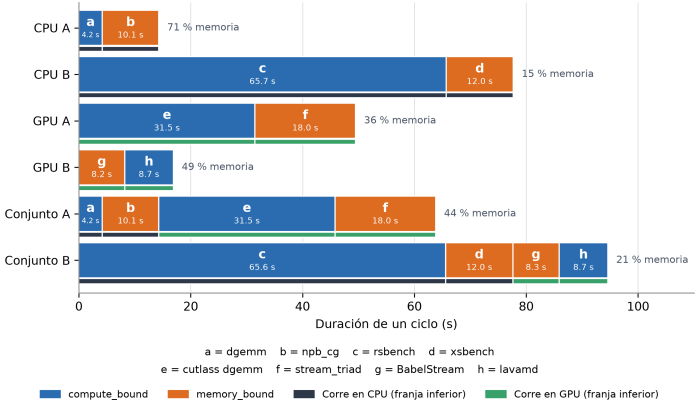
Premisa física

Con el reloj fijado, *compute* escala con el reloj y *memory* no cae hasta 510 MHz

Permanencia mínima GPU

3.7 s: diez veces la latencia de un cambio completo

A14 · Aplicaciones compuestas y clasificación en el lazo

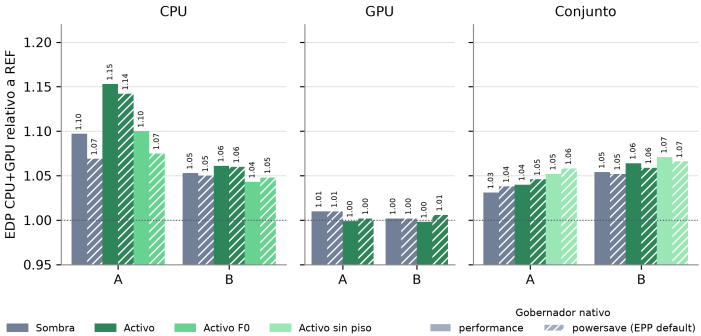


Tiempo en fases *memory*: de 15 % (B, CPU) a 71 % (A)

CPU: acierta 96.7–97.5 % (vistas) y 99.6–99.8 % (inéditas), ponderado por ventanas

GPU: ninguna decisión errónea; se abstuvo en 2/12 (A) y 4–5/12 (B)

A15 · Matriz inicial: el proceso de CPU empeora el EDP

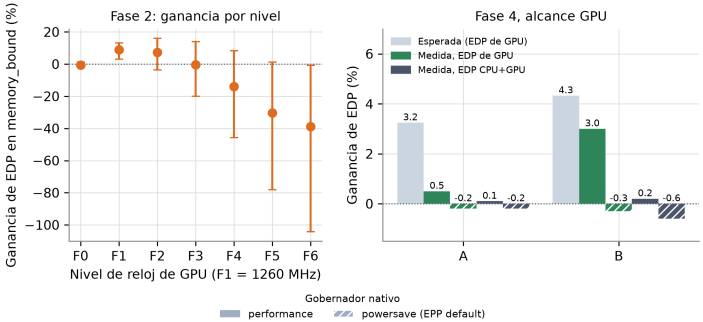


GPU activo: EDP 1.00 en A y B; sombra 1.01 y 1.00

CPU activo: 1.15 (A) y 1.06 (B); conjunto 1.04 y 1.06 ($p = 0.0079$)

Sombra CPU ya en 1.10 y 1.05: +13.6 W y +6.2 W a igual duración

A16 · Potencial de la GPU en la matriz inicial

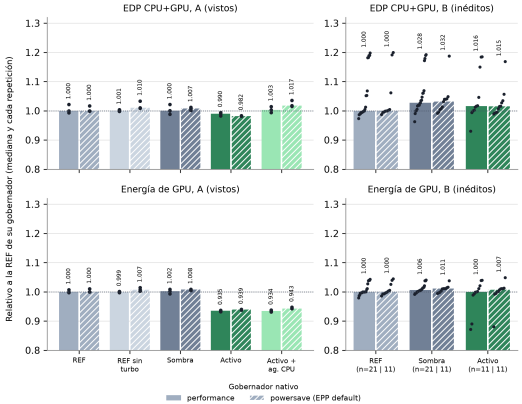


Esperado (fracción *memory* × 8.9 %): 3.2 % (A) y 4.3 % (B) de EDP GPU

Medido: 0.5 % y 3.0 % de EDP GPU; 0.1 y 0.2 % del EDP CPU+GPU; no se separa de cero

En B, 4 de 6 decisiones sobre BabelStream fueron abstenciones: motivó el escenario E

A17 · Escenario E: fases de memoria largas

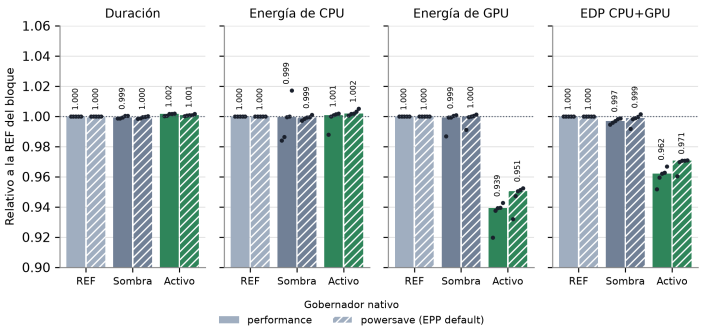


E-A: agente 0.962 de EDP CPU+GPU (IC95 0.946–0.977); energía GPU 0.928

F1 fijo 0.964; agente/F1 fijo = 0.998 (IC95 0.990–1.006)

E-B: sombra 1.028 y activo 1.016 (descriptivo); myocyte bimodal

A18 · CloverLeaf: una decisión sostenida en una aplicación externa

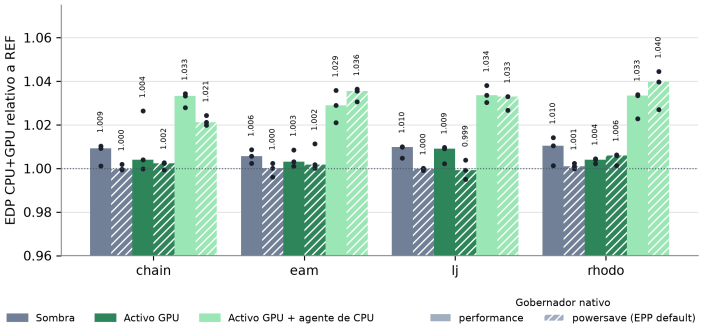


Una decisión *memory-bound* con confianza 1.00 a los ~3 s; resultado físico idéntico (10^{-8})

EDP CPU+GPU 0.961 (IC95 0.954–0.968); energía GPU 0.936; duración 1.001

Se comporta como F1 fijo mientras la GPU está activa; F1 fijo no se midió aquí

A19 · LAMMPS: paridad con la base



GPU activo entre 1.003 y 1.009 de la base según la entrada

chain y rhodo: sin decisiones; lj y eam: solo *compute* → nunca F1

Con el agente de CPU en observación: +2.9 a 3.4 % de EDP